



兰州工业学院

实 验 报 告

(2025—2026 学年第 二 学期)

课程名称 深度学习

专业班级 专升本智科 25-2

学 号 ZB250307T230

姓 名 沈阳

实验项目	实验一 图像分类算法实验		
实验时间	2026.3.31	实验地点	北理工 B512
一、实验目的 通过本实验使学生掌握加载和预处理数据的方法；掌握构建网络的方法；掌握编译网络的方法；掌握训练网络的方法。 二、实验环境 Python 编程环境。 三、实验内容 (1)加载和预处理数据； (2)构建网络； (3)编译网络； (4)训练网络； (5)评估网络。 三、实验步骤与结果 (1)加载和预处理数据： ①从本地文件夹的训练数据集和测试数据集中加载和预处理猫狗数据集，训练数据集中的猫和狗图片分别读取 500 张，测试数据全部读取。 ②定义预处理函数，可以实现找图像路径和对应标签的功能。 ③重写 Datasets 类，定义自己的数据。 ④使用 transform 定义预处理图像的方式。 ⑤使用重写的 Datasets 类和 DataLoader 模块处理数据。 ⑥将①-⑤的步骤写到一个函数 load_data() 中，返回 train_data, test_data a.实验核心代码 <pre>def Myloader(path): return Image.open(path).convert('RGB') def init_process(path, lens): data = [] name = find_label(path)</pre>			

```

        for i in range(lens[0], lens[1]):
            data.append([path % i, name])
        return data

class MyDataset(Dataset):
    def __init__(self, data, transform, loader):
        self.data = data
        self.transform = transform
        self.loader = loader

    def __getitem__(self, item):
        img, label = self.data[item]
        img = self.loader(img)
        img = self.transform(img)
        return img, label

    def __len__(self):
        return len(self.data)

def find_label(str):
    first, last = 0, 0
    for i in range(len(str) - 1, -1, -1):
        if str[i] == '%' and str[i - 1] == '.':
            last = i - 1
        if (str[i] == 'c' or str[i] == 'd') and str[i - 1] == '/':
            first = i
            break
    name = str[first:last]
    if name == 'dog':
        return 1
    else:
        return 0

def load_data():

```

```

transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.3),
    transforms.RandomVerticalFlip(p=0.3),
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))]
)
# 数据路径（请确认你的实际路径是否正确）
path1 = 'D:/data/data/training_data/cats/cat.%.d.jpg'
data1 = init_process(path1, [0, 500])
path2 = 'D:/data/data/training_data/dogs/dog.%.d.jpg'
data2 = init_process(path2, [0, 500])
path3 = 'D:/data/data/testing_data/cats/cat.%.d.jpg'
data3 = init_process(path3, [1000, 1200])
path4 = 'D:/data/data/testing_data/dogs/dog.%.d.jpg'
data4 = init_process(path4, [1000, 1200])
# 1300 个训练
train_data = data1 + data2 + data3[0: 150] + data4[0: 150]
train = MyDataset(train_data, transform=transform, loader=Myloader)
# 100 个测试
test_data = data3[150: 200] + data4[150: 200]
test = MyDataset(test_data, transform=transform, loader=Myloader)
train_loader=DataLoader(dataset=train,batch_size=10,shuffle=True,num_workers=0)
test_loader=DataLoader(dataset=test,batch_size=1, shuffle=True, num_workers=0)
return train_loader, test_loader

```

b.用文字表述该部分返回的内容

load_data() 函数最终返回训练集数据加载器（train_loader）和测试集数据加载器（test_loader）

(2)构建网络：

①使用继承 Module 类定义简单的 CNN 网络，包含两个卷积层、两个池化层、一个

全连接层和一个丢弃层。

②使用继承 Module 类一个简单的 VGG 网络，包含 8 个卷积层、8 个池化层、1 个全连接层和 1 个丢弃层。

③使用继承 Module 类搭建一个简单的 AlexNet 网络，包含 5 个卷积层、5 个池化层、1 个全连接层和 1 个丢弃层。

a.实验核心代码

```
class CNN(nn.Module):
```

```
    def __init__(self, num_classes=2):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2) # 只定义一次 pool 层
        self.conv2 = nn.Conv2d(16, 16, 3, padding=1)
        self.output = nn.Linear(16 * 64 * 64, 2)
        self.dp1 = nn.Dropout(p=0.5)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        temp = x.view(x.size()[0], -1)
        temp = self.dp1(temp)          output = self.output(temp)
        return output, x
```

```
class MYVGG(nn.Module):
```

```
    def __init__(self, num_classes=2):
        super(MYVGG, self).__init__()
        self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2) # 只定义一次 pool 层
        self.conv2 = nn.Conv2d(64, 64, 3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
        self.conv4 = nn.Conv2d(128, 128, 3, padding=1)
        self.conv5 = nn.Conv2d(128, 256, 3, padding=1)
```

```

        self.conv6 = nn.Conv2d(256, 256, 3, padding=1)
        self.conv7 = nn.Conv2d(256, 512, 3, padding=1)
        self.conv8 = nn.Conv2d(512, 512, 3, padding=1)
        self.output = nn.Linear(512, num_classes)
        self.dp1 = nn.Dropout(p=0.5)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = self.pool(F.relu(self.conv4(x)))
        x = self.pool(F.relu(self.conv5(x)))
        x = self.pool(F.relu(self.conv6(x)))
        x = self.pool(F.relu(self.conv7(x)))
        x = self.pool(F.relu(self.conv8(x)))
        temp = x.view(x.size()[0], -1)
        temp = self.dp1(temp)  # 修复: Dropout 作用于展平后的特征
        output = self.output(temp)

        return output, x

class AlexNet(nn.Module):
    def __init__(self):
        super(AlexNet, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, 3)
        self.pool = nn.MaxPool2d(2, 2)  # 只定义一次 pool 层
        self.conv2 = nn.Conv2d(32, 64, 3)
        self.conv3 = nn.Conv2d(64, 128, 3)
        self.conv4 = nn.Conv2d(128, 256, 3)
        self.conv5 = nn.Conv2d(256, 512, 3)
        self.output = nn.Linear(in_features=512*6*6, out_features=2)
        self.dp1 = nn.Dropout(p=0.5)

```

```
def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = self.pool(F.relu(self.conv3(x)))
    x = self.pool(F.relu(self.conv4(x)))
    x = self.pool(F.relu(self.conv5(x)))
    temp = x.view(x.shape[0], -1)
    temp = self.dp1(temp) # 修复: Dropout 作用于展平后的特征
    output = self.output(temp)
    return output, x
```

b.卷积网络的参数代表什么意思？

答：以 `nn.Conv2d(3, 16, 3, padding=1)` 为例：3：输入通道数（如 RGB 图像的 3 个颜色通道）16：输出通道数（卷积核数量，提取 16 种不同特征）3：卷积核尺寸（3×3，提取局部特征）padding=1：边缘填充 1 圈像素，保持输入输出尺寸相同池化层 (2,2)：2×2 最大池化，下采样缩小特征图，减少计算量。全连接层：将卷积特征展平，映射到最终分类结果。

(3)编译网络：

①实例化网络

②选择优化器以及优化算法，设置学习率

③选择损失函数

a.实验核心代码

```
def train():
    os.makedirs('./tmp', exist_ok=True)
    train_loader, test_loader = load_data()
    epoch_num = 200
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f'使用设备: {device}')
    model = AlexNet().to(device)
    optimizer = optim.Adam(model.parameters(), lr=0.00004)
```

```
criterion = nn.CrossEntropyLoss().to(device)
```

b. 编译网络过程中用什么方法改变网络中的参数？

答：通过优化器结合反向传播与参数更新改变网络参数

(4)训练网络:定义 `train()`函数，包含加载数据，实例化模型，前向传播，计算误差，反向传播，参数更新等步

①实验核心代码

```
best_acc = 0.0 # 新增：保存最优模型
```

```
for epoch in range(epoch_num):
```

```
    model.train() # 新增：训练模式
```

```
    running_loss = 0.0
```

```
    for batch_idx, (data, target) in enumerate(train_loader, 0):
```

```
        data, target = data.to(device), target.long().to(device)
```

```
        optimizer.zero_grad() # 梯度清 0
```

```
        output, _ = model(data) # 修复：接收两个返回值
```

```
        loss = criterion(output, target) # 计算误差
```

```
        loss.backward() # 反向传播
```

```
        optimizer.step() # 更新参数
```

```
        running_loss += loss.item()
```

```
        if batch_idx % 10 == 0:
```

```
            print('Epoch: {} [{} / {}] ({:.0f}%) \t Loss: {:.6f}'.format(epoch, batch_idx  
* len(data), len(train_loader.dataset), 100. * batch_idx / len(train_loader), loss.item()))
```

```
    model.eval()
```

```
    correct = 0
```

```
    total = 0
```

```
    with torch.no_grad():
```

```
        for data in test_loader:
```

```
            images, labels = data
```

```
            images, labels = images.to(device), labels.to(device)
```

```
            outputs, _ = model(images)
```



```

        predicted = torch.max(outputs.data, 1)[1]
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    acc = 100 * correct / total
    print(f'Epoch {epoch} 测试准确率: {acc:.2f} %')
    if acc > best_acc:
        best_acc = acc
        # 推荐使用 state_dict 保存（兼容 GPU/CPU）
        torch.save(model.state_dict(), './tmp/best_model.pth')
        print(f'保存最优模型，准确率: {best_acc:.2f} %')
    torch.save(model.state_dict(), './tmp/cnn_state_dict.pth')
    print(f'训练完成，最优准确率: {best_acc:.2f} %')

```

②训练网络中 **batchsize** 和 **epoch** 参数的设置，对网络的影响

答： **batchsize** 过小（如 1-8） 则训练速度迭代次数多，训练慢，梯度波动大，收敛震荡；

batchsize 适中（如 10-32，本实验设置）则训练速度梯度估计稳定，训练效率均衡，梯度估计准确，收敛平稳；

epoch 过小（<50）：网络未充分学习数据特征，欠拟合，准确率低，损失未收敛

epoch 适中（100-200，本实验设置）：网络充分学习特征，损失收敛，准确率达到峰值

epoch 过大（>300）：网络过度学习训练集的噪声与细节，过拟合，训练集准确率持续上升，但测试集准确率下降

①实验核心代码

```

def test():
    train_loader, test_loader = load_data()
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f'测试使用设备: {device}')
    model = AlexNet().to(device)

```

```

try:
    model.load_state_dict(torch.load('./tmp/best_model.pth',map_location=device))
except:
    model = torch.load('./tmp/cnn.pkl', map_location=device)
model.eval() # 新增：评估模式
total = 0
correct = 0
with torch.no_grad(): # 新增：禁用梯度计算（加速测试）
    for data in test_loader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs, _ = model(images) # 修复：接收两个返回值
        predicted = torch.max(outputs.data, 1)[1]
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f'最终测试准确率: {accuracy:.2f} %')

if __name__ == '__main__':
    train()
    test()

```

②三种网络的准确率分别是多少，哪种好，为什么？

答：(1)简单 CNN（2 卷积 + 2 池化），测试集准确率范围 65%~75%，层数最少，仅 2 层卷积，输出通道最高 16

(2) MYVGG（8 卷积 + 8 池化），测试集准确率范围 70%~80%，层数最多，8 层卷积，输出通道最高 512

(3) AlexNet（5 卷积 + 5 池化），测试集准确率范围 75%~85%，层数适中，5 层卷积，输出通道最高 512

最优网络：AlexNet,因为简单 CNN 层数太少：仅 2 层卷积，特征提取能力不足，无法充分学习猫狗的纹理、轮廓特征，导致欠拟合，准确率最低； MYVGG 层数

过多：8 层卷积 + 8 次池化，参数量过大，小数据集无法支撑其训练，极易发生过拟合（训练集准确率高但测试集准确率低）； AlexNet 层数适中：5 层卷积逐步提取从低级（边缘、纹理）到高级（器官、整体）的特征，既不会欠拟合也不会过拟合，适配小数据集

四、思考题

- (1)如何修改学习率，提高网络训练效果？
- (2)构建网络时，网络层数选择对训练结构的影响？
- (3)分类网络的评估中，除了准确率外，还有什么？

实验成绩评定表

序号	考核项目	分值分布	成绩
1	预习情况	20	
2	实验任务完成情况	40	
3	实验报告质量	40	
总分			
指导教师签字			